Alliance University

IT ARVR

Linux Administrative System (E1ITA307)

LINUX INFRASTRUCTURE DEPLOYMENT PROJECT REPORT

On

LINUX STORAGE MONITORING SYSTEM

Submitted by

Aman N (2411021061141)
M Nadir (L2411021061698)
K S Prajwal (2411021061418)
Rabi Kant (2411021061178)

In partial fulfilment of the academic requirements for
IT ARVR

Under the supervision of
Prof. U. Ananthanagu
Assistant Professor

2026–27
02-11-2026

IT ARVR

Alliance University

CERTIFICATE TEMPLATE — NOT AN APPROVAL

This page is an editable certificate template only. The authorized faculty and institution must review and complete the signature fields. No approval, signature, or certification is asserted by this document.

Project title: **Linux Storage Monitoring System**

Student name(s): Aman N; M Nadir; K S Prajwal; Rabi Kant

Register number(s): 2411021061141; L2411021061698; 2411021061418; 2411021061178

Course and academic year: Linux Administrative System (E1ITA307); 2026–27

Authorized faculty supervisor: Prof. U. Ananthanagu, Assistant Professor Signature and date: [TO BE COMPLETED BY AUTHORIZED FACULTY]

Course coordinator / department authority: [OPTIONAL INSTITUTIONAL FIELD] Signature and date: [TO BE COMPLETED BY AUTHORIZED PERSON]

TABLE OF CONTENTS

1. INTRODUCTION 6

1.1 Aim 6

1.2 Background 6

1.3 Problem Statement 6

1.4 Objectives 6

2. METHODOLOGY 7

2.1 Tools and Technologies Used 7

2.1.1 Utilities Used by the Implementation 7

2.2 Approach / Development Process 7

3. LINUX SYSTEM ADMINISTRATION — DESIGN AND IMPLEMENTATION 8

3.1 System Architecture 8

3.2 Linux Environment Setup 8

3.3 User and Group Management 8

3.4 File and Directory Management 9

3.5 Package and Process Management 9

3.6 Storage and File System Configuration 10

3.7 Backup and Automation 10

3.8 Security and Access Control 10

4. RESULTS AND TESTING 11

4.1 Implementation Results 11

4.2 Testing and Validation 11

5. CONCLUSION AND FUTURE SCOPE 12

5.1 Conclusion 12

5.2 Current Limitations and Future Scope 12

REFERENCES 13

Technical References 13

APPENDICES 14

Appendix A. Installation and Execution Commands 14

Appendix B. Configuration Example 14

Appendix C. Selected Source Excerpts 14

Appendix D. Recorded Test Transcript 14

Appendix E. Screenshot Evidence Placeholders 15

LIST OF FIGURES

Figure 3.1: System architecture derived from the inspected implementation 8

Figure 3.2: Audited project directory structure 9

Figure 4.1: 4.1 Ubuntu terminal environment (capture on target host) 15

Figure 4.2: 4.2 Project directory structure (capture on target host) 15

Figure 4.3: 4.3 Installation process (capture on target host) 15

Figure 4.4: 4.4 Disk monitoring output (capture on target host) 15

Figure 4.5: 4.5 Inode monitoring output (capture on target host) 15

Figure 4.6: 4.6 Storage history and growth sample (capture on target host) 15

Figure 4.7: 4.7 Warning and critical alert output (capture on target host) 16

Figure 4.8: 4.8 Generated text report (capture on target host) 16

Figure 4.9: 4.9 Cron configuration and execution log (capture on target host) 16

Figure 4.10: 4.10 Timestamped backup creation (capture on target host) 16

Figure 4.11: 4.11 Compressed archive integrity check (capture on target host) 16

Figure 4.12: 4.12 Selected-file recovery (capture on target host) 16

LIST OF TABLES

Table 2.1: Utilities used by the implementation 7

Table 3.1: Project directory and file roles 9

Table 4.1: Test and validation results 11

1. INTRODUCTION

1.1 Aim

The aim of this project is to provide an auditable Bash toolset for monitoring mounted filesystem block capacity and inode use, recording repeated measurements, raising threshold-based administrator alerts, and producing a text summary. The implementation is project-scoped and intended to run as the ordinary account that owns the files.

1.2 Background

Linux storage capacity has at least two operational dimensions. Data blocks hold file contents, while inodes hold file metadata and directory references. A filesystem may have available bytes but too few free inodes to create more files. Conversely, high block consumption can prevent writes even when inode capacity remains. A single manual command offers a snapshot; a scheduled sample history helps reveal whether use is changing.

The supplied reference report uses Bash and standard Linux utilities to illustrate these tasks. This report follows its chapter order and academic presentation while describing the scripts and test evidence in the current workspace.

1.3 Problem Statement

Administrators need a repeatable way to inspect filesystem capacity and inode consumption, compare values with warning and critical limits, retain dated observations, and review reports and alerts. The delivered project implements those mechanisms. Linux collection and report generation have now been exercised in Kali Linux on WSL 2; cron timing and Linux permission modes remain unverified.

1.4 Objectives

The objectives are to collect disk and inode usage from mounted filesystems; filter configured pseudo-filesystem types; compare percentages against separate disk and inode thresholds; write timestamps and capacity figures to CSV; compare used bytes with the previous sample; keep monitor and administrator alert logs; produce a timestamped text report; install per-user cron jobs; and create, verify, and recover a compressed project archive.

2. METHODOLOGY

2.1 Tools and Technologies Used

The scripts target Ubuntu or a compatible Debian-derived Linux. Bash provides strict-mode execution, functions, arrays, arithmetic comparisons, process substitution, and configuration parsing. GNU df reports filesystem capacity and inode counts; stat identifies filesystem types; awk reads the previous CSV sample; du, sort, and tail prepare a best-effort directory-size ranking in the report. date supplies timestamps; tee appends log entries while displaying alerts; tar and gzip create compressed backups; realpath and symbolic-link checks constrain recovery; cron and crontab provide per-user scheduling.

The installer also uses find and chmod to set script permissions. grep is used when installing marked cron lines; ls is used in test and operator commands. sed appears in the dependency guidance but is not called by the project scripts. CSV is a plain comma-separated history file, not a database. Python, python-docx, ReportLab, and Pillow were used to prepare the documentation artifacts; they are not runtime dependencies of the monitor.

2.1.1 Utilities Used by the Implementation

Table 2.1 distinguishes commands used by project code from those used only in operator instructions or package guidance.

Table 2.1: Utilities used by the implementation

Utility	Observed use in project
bash	Module language, strict mode, control flow, arithmetic, and arrays
df	Block capacity and inode usage; used by monitoring and report commands
stat	Filesystem type for configured exclusion filtering
awk	Lookup of previous mountpoint usage in CSV history
du / sort / tail	Best-effort top-level size ranking in generated report
date / tee / touch / mkdir	Timestamps, append/display logging, file and directory initialization
find / chmod / ls	Installer file modes; ls is used in test/operator commands
grep / crontab / mktemp	Marked cron line replacement in the cron installer
tar / gzip / realpath	Backup archive, archive listing, resolved-path restrictions, recovery
sed	Installed by package guidance but not invoked by the project scripts

2.2 Approach / Development Process

The workspace implementation is organized around a shared library and small command-line modules. First, the requested behavior was mapped to disk collection, inode collection, sampling, alerts, reports, cron, and backup/recovery. Configuration was separated from code. The scripts were then connected through run_all.sh and monitor_cycle.sh. A safe test script checks parser syntax, threshold boundaries, alert branches, live collection and report generation on Linux, tar listing, a project-local recovery copy, and unsafe destination rejection. It detects MSYS and skips Linux-only collection checks rather than treating Windows volumes as Linux evidence.

The final suite ran under Kali Linux in WSL 2 and reported 12 passed, 0 failed. It exercised live disk/inode collection, CSV sampling, report generation, alerts, backup integrity, recovery, and traversal rejection. Cron timing and installer permission modes were not part of this suite.

3. LINUX SYSTEM ADMINISTRATION — DESIGN AND IMPLEMENTATION

3.1 System Architecture

Figure 3.1 shows the control and data paths visible in the source. The Linux host supplies filesystem statistics. Disk, inode, and growth scripts gather those statistics; common.sh parses thresholds and emits alerts; growth samples are appended to CSV; the report combines df, du, alert, and history output. Cron invokes a five-minute monitoring cycle and a fifteen-minute report command. Backup and recovery are separate project-local commands. The diagram represents code structure, not a captured runtime.

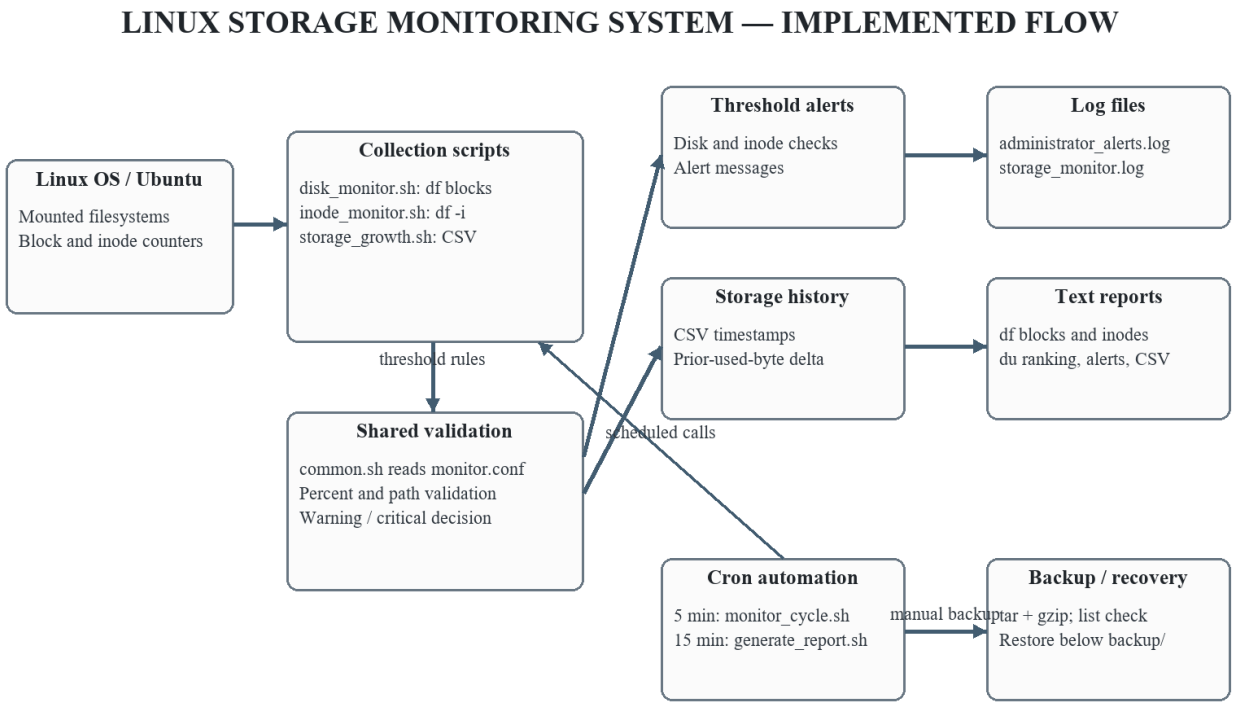


Figure 3.1: System architecture derived from the inspected implementation

3.2 Linux Environment Setup

The target is Ubuntu 20.04 or newer, as stated by the project README. The installer reads /etc/os-release and accepts Ubuntu or an ID_LIKE value containing Debian. It checks for Bash, df, du, find, ls, awk, sed, grep, date, tar, gzip, stat, sort, tail, and hostname. If utilities are missing, it prints an apt installation command; it does not automatically install packages.

The test host was Kali Linux running in WSL 2 on Windows. This provides Linux execution evidence, but it is not an Ubuntu installation. The installer, cron timing, and configured permission modes were not tested. The companion guide contains the exact extraction, dependency, installer, and first-run commands.

3.3 User and Group Management

No project-specific Linux user or group is created. The code runs as the invoking account. Cron entries are installed in that account’s personal crontab, so the account must continue to own and access the project. The project has no shared-directory or group-membership demonstration; report descriptions of user/group configuration are therefore limited to this fact.

3.4 File and Directory Management

The repository separates executable modules, configuration, data, logs, reports, backups, tests, screenshots, and documentation. The Kali WSL test produced a CSV sample, test logs, and a timestamped runtime report. The backup directory contains a verified archive and a recovery copy produced by the integration test.

Figure 3.2 and Table 3.1 summarize the project layout. The archive and recovery copy are test artifacts; documentation placeholders are not represented as collected screenshots.

```
linux-storage-monitor/
■■■ bin/
■   ■■■ disk_monitor.sh
■   ■■■ inode_monitor.sh
■   ■■■ storage_growth.sh
■   ■■■ alert_monitor.sh
■   ■■■ generate_report.sh
■   ■■■ monitor_cycle.sh
■   ■■■ run_all.sh
■   ■■■ install_cron.sh
■   ■■■ backup_project.sh
■   ■■■ restore_backup.sh
■   ■■■ lib/common.sh
■■■ config/monitor.conf
■■■ data/storage_history.csv  (Kali WSL sample)
■■■ logs/  (test run and helper alerts)
■■■ reports/  (generated storage report)
■■■ backup/  (verified test archive and recovery copy)
■■■ tests/run_tests.sh, test_results.txt
■■■ screenshots/README.md  (capture instructions; no captures yet)
■■■ tools/ (documentation generators)
■■■ documentation/ (this report and guides)
■■■ install.sh / README.md / EXECUTION_GUIDE.md
■■■ system_architecture.png
```

Figure 3.2: Audited project directory structure

Table 3.1: Project directory and file roles

Path	Purpose / present state
bin/	Bash modules and shared library
config/monitor.conf	Thresholds, exclusions, and relative output locations
data/storage_history.csv	Kali WSL sample recorded by integration test
logs/	Test run and helper alert output
reports/	Generated timestamped storage report
backup/	Verified test archive and restored project copy
tests/	Integration test source and actual test transcript
screenshots/	Instructions only; no captures collected
documentation/	Final report and companion guides

3.5 Package and Process Management

The installer checks for command availability and reports apt guidance if required utilities are missing. It does not install packages, enable services, or create a long-running process. Interactive commands exit after a scan. Cron is optional and invokes short-lived scripts. The cron installer rewrites only lines tagged with LINUX_STORAGE_MONITOR and retains other crontab lines. Cron service status and runtime dispatch have not been checked on Ubuntu.

3.6 Storage and File System Configuration

The project does not partition disks, create filesystems, resize logical volumes, mount devices, or change fstab. disk_monitor.sh uses df -P -B1 for byte totals and percentages; inode_monitor.sh uses df -P -i. Both remove the percent suffix, identify filesystem type with stat -f -c %T, skip types in EXCLUDED_FILESYSTEM_TYPES, print a table, and invoke the common threshold comparison. These commands operate on available mount statistics and do not intentionally create storage pressure.

The configured defaults are 75% warning and 90% critical for both block and inode use. Values are integer percentages from 0 to 100; warning must be lower than critical. The output and alerts are per mounted filesystem. The Kali WSL integration run collected disk and inode statistics and saved them in the generated runtime report; these measurements are specific to the WSL environment.

```
df -P -B1     # bytes: total, used, available, percentage
df -P -i       # inode totals, use, free count, percentage
stat -f -c %T -- "$mount"   # filesystem type used for exclusions
```

3.7 Backup and Automation

storage_growth.sh initializes the CSV header if the file is absent or empty. For each included mountpoint, it appends timestamp, filesystem, mountpoint, total/used/available bytes, usage percent, and inode totals/used/free/percent. It then uses awk to find the prior CSV row for that mountpoint and prints the signed used-byte difference. The test run verified that the CSV contains a header and at least one sample.

install_cron.sh adds a monitor_cycle.sh entry every five minutes and a generate_report.sh entry every fifteen minutes. Both append output to logs/cron.log. Cron installation and scheduled execution were not tested.

generate_report.sh writes a timestamped report with df -hP, df -iP, a best-effort du ranking of top-level directories on the root filesystem, the last 50 alert lines, and the last 20 CSV lines. The Kali WSL integration run generated a runtime report in reports/.

backup_project.sh creates a gzip tar archive and checks that it can be listed. The Kali WSL test passed archive integrity, recovered selected files into backup/test-recovery, and confirmed unsafe destination rejection.

```
timestamp,filesystem,mountpoint,total_bytes,used_bytes,available_bytes,usage_percent,inodes_total,inodes_used,inode
```

```
*/5 * * * * /path/to/linux-storage-monitor/bin/monitor_cycle.sh >> /path/to/linux-storage-monitor/logs/cron.log 2>&
*/15 * * * * /path/to/linux-storage-monitor/bin/generate_report.sh >> /path/to/linux-storage-monitor/logs/cron.log
```

3.8 Security and Access Control

Scripts use set -Eeuo pipefail. monitor.conf is parsed as key/value data and is not sourced as executable shell code. The loader validates numeric thresholds, threshold ordering, report top-N range, filesystem-type syntax, and relative output paths. The restore command checks argument count, archive location, destination syntax, symbolic-link components, and the final resolved destination.

The installer attempts mode 750 for project directories and scripts, and mode 640 for configuration, CSV, and logs. Several chmod operations are followed by || true, which can suppress a permission-setting failure; the exact modes were not verified on Ubuntu. Normal monitoring does not require root. Package installation and cron service startup use sudo only when the operator chooses those system-level setup steps. A hostile archive placed in the backup directory is outside the tested threat model; only project-generated archives were used.

```
check_threshold() {
  local metric=$1 mount=$2 pct=$3 warning=$4 critical=$5
  if (( pct >= critical )); then alert CRITICAL "$metric" "$mount" "$pct" "$critical"
  elif (( pct >= warning )); then alert WARNING "$metric" "$mount" "$pct" "$warning"
  fi
}
```

4. RESULTS AND TESTING

4.1 Implementation Results

The Kali WSL test run executed the full Linux integration path: disk and inode collection, CSV sampling, and timestamped report generation all passed. Threshold boundary checks, warning/critical alerts, Bash parsing, compressed backup integrity, selected-file recovery, and unsafe-path rejection also passed. The complete suite reported 12 passed, 0 failed.

The runtime report and test log are saved under reports/ and logs/. Cron timing, Linux permission modes, and terminal screenshots remain unverified. WSL output demonstrates Kali Linux execution and is not presented as an Ubuntu screenshot or Ubuntu host result.

4.2 Testing and Validation

Table 4.1 reports the test case, objective, command, expected result, actual result, and status. The suite was launched in Kali Linux under WSL 2 with `./tests/run_tests.sh`; its actual output is included in Appendix D and tests/test_results.txt. Cron timing, permission installation, and screenshot capture were not covered by this run.

Table 4.1: Test and validation results

Test Case	Objective	Command	Expected Result	Actual Result	Status
Source audit	Check report against Project 7 and source	Inspect PDFs and project tree	Report matches selected project and code	Guideline p.15; reference and implementation audited	PASS
Bash syntax	Parse shell modules and tests	bash -n *.sh	No syntax errors	All parser checks passed	PASS (Kali WSL)
Threshold bounds	Check 0/100 and reject 101	tests/run_tests.sh	Endpoints accepted; 101 rejected	Regex passed; full config mutation not tested	PASS (limited)
Disk and inode	Collect mounted filesystem capacity	tests/run_tests.sh (run_all.sh)	Tables and threshold checks	Full disk/inode/report cycle completed	PASS (Kali WSL)
CSV / growth	Record repeated samples and delta	tests/run_tests.sh (run_all.sh)	CSV header and at least one sample	CSV header and sample verified	PASS (Kali WSL)
Alert thresholds	Exercise warning and critical branches	Test helper at 75% / 90%	Both alert levels logged	Expected TEST-MOUNT strings found	PASS (helper)
Run-all and report	Run scan/history/report sequence	tests/run_tests.sh (run_all.sh)	Tables, CSV, and report files	Cycle completed; CSV sample and report found	PASS (Kali WSL)
Cron	Verify five- and fifteen-minute jobs	install_cron.sh; crontab -l	Marked jobs and cron log entries	No Ubuntu cron evidence	NOT EXECUTED
Backup integrity	Create and list gzip tar	backup_project.sh; tar -tzf	Archive listing succeeds	Archive integrity check passed	PASS (Kali WSL)
Recovery and path safety	Restore files; reject traversal	restore_backup.sh archive destination	README recovered; outside path refused	Both checks passed	PASS (Kali WSL)
Permissions / screenshots	Verify modes and collect evidence	stat; capture 12 views	Configured modes and genuine PNG evidence	Neither executed on Ubuntu	NOT EXECUTED

5. CONCLUSION AND FUTURE SCOPE

5.1 Conclusion

The implementation demonstrates a modular approach to Linux storage administration: capacity and inode collection, configured thresholds, append-only CSV history, log-based alerts, readable reports, scheduled execution, and compressed backup/recovery. The shared configuration and explicit user-level execution keep the toolset small and inspectable. The Kali WSL evidence verifies the principal Linux collection and reporting path; cron timing, permission modes, and Ubuntu-specific behavior remain to be demonstrated.

5.2 Current Limitations and Future Scope

The current release has no email or webhook alert transport, retention policy, charting, daemon, database, or dashboard. CSV fields are emitted as simple comma-separated values rather than a full RFC-compliant quoting routine. Directory-size ranking scans the top level of / and is best-effort; permission errors are discarded. Test coverage skips live collection on MSYS and does not execute cron or permission installation. Future work may add robust CSV escaping, configurable scan roots and retention, tested systemd timer support, alert delivery, trend visualization, and encrypted off-host backups.

REFERENCES

Technical References

1. GNU Project, GNU Bash Reference Manual, https://www.gnu.org/software/bash/manual/ (accessed 2 November 2026).

2. GNU Project, GNU Coreutils Manual, sections for df, du, stat, date, sort and tail, https://www.gnu.org/software/coreutils/manual/coreutils.html (accessed 2 November 2026).

3. GNU Project, GNU Findutils Manual, https://www.gnu.org/software/findutils/manual/find.html (accessed 2 November 2026).

4. Ubuntu Manpages, crontab(5), https://manpages.ubuntu.com/manpages/jammy/man5/crontab.5.html (accessed 2 November 2026).

5. Ubuntu Manpages, crontab(1), https://manpages.ubuntu.com/manpages/resolute/man1/crontab.1.html (accessed 2 November 2026).

6. Linux System Administration Hackathon Based DSA-SEE Guidelines, 09.09.26, pp. 14–15, user-supplied PDF (Project 7 description and evaluation context).

7. Linux new.pdf, user-supplied 75-page report, used as a structure and presentation reference. No identities, signatures, or approval content are reproduced.

APPENDICES

Appendix A. Installation and Execution Commands

The exact beginner sequence is provided in documentation/README_Execution_Guide.md. It covers package setup, extracting the transfer ZIP, chmod, installer, run_all.sh, individual modules, cron inspection, report/log inspection, and backup/recovery.

Appendix B. Configuration Example

The following values are read from config/monitor.conf. They are data values; the file is not sourced as shell code.

```
WARNING_THRESHOLD=75
CRITICAL_THRESHOLD=90
INODE_WARNING_THRESHOLD=75
INODE_CRITICAL_THRESHOLD=90
REPORT_TOP_N=10
EXCLUDED_FILESYSTEM_TYPES=tmpfs,devtmpfs,squashfs,overlay
ALERT_LOG=logs/administrator_alerts.log
MONITOR_LOG=logs/storage_monitor.log
HISTORY_CSV=data/storage_history.csv
REPORT_DIR=reports
BACKUP_DIR=backup
```

Appendix C. Selected Source Excerpts

Listing C.1 shows the actual threshold decision. The alert helper writes the message to both configured logs and stderr.

Listing C.2 is the CSV header emitted by init_history().

Listing C.3 shows the cron frequencies installed by install_cron.sh.

Listing C.1: Threshold decision function

```
check_threshold() {
  local metric=$1 mount=$2 pct=$3 warning=$4 critical=$5
  if (( pct >= critical )); then alert CRITICAL "$metric" "$mount" "$pct" "$critical"
  elif (( pct >= warning )); then alert WARNING "$metric" "$mount" "$pct" "$warning"
  fi
}
```

Listing C.2: CSV schema

```
timestamp,filesystem,mountpoint,total_bytes,used_bytes,available_bytes,usage_percent,inodes_total,inodes_used,inode
```

Listing C.3: Cron schedule template

```
*/5 * * * * /path/to/linux-storage-monitor/bin/monitor_cycle.sh >> /path/to/linux-storage-monitor/logs/cron.log 2>&
*/15 * * * * /path/to/linux-storage-monitor/bin/generate_report.sh >> /path/to/linux-storage-monitor/logs/cron.log
```

Appendix D. Recorded Test Transcript

Command run from the project root in Kali Linux WSL 2: `./tests/run_tests.sh`. The transcript below is the captured output from that run.

```
PASS: configuration file readable
PASS: main runner executable
PASS: threshold parser accepts zero and 100
PASS: threshold parser rejects 101
PASS: monitor scripts pass Bash parser
PASS: disk, inode, growth and report cycle completes
PASS: CSV header and at least one sample
PASS: generated report exists
PASS: warning and critical alert branches execute
PASS: compressed backup passes tar integrity check
PASS: selected project files recover
PASS: unsafe destination rejected

Test summary: 12 passed, 0 failed
```

Appendix E. Screenshot Evidence Placeholders

No terminal screenshot is fabricated. The following figure slots must be replaced with genuine Ubuntu captures after the commands in README_Execution_Guide.md are run. Each caption describes the evidence to collect.

SCREENSHOT PLACEHOLDER — NOT CAPTURED

Insert a genuine Ubuntu terminal capture for this figure. See README_Execution_Guide.md for the exact command.

Figure 4.1: 4.1 Ubuntu terminal environment (capture on target host)

SCREENSHOT PLACEHOLDER — NOT CAPTURED

Insert a genuine Ubuntu terminal capture for this figure. See README_Execution_Guide.md for the exact command.

Figure 4.2: 4.2 Project directory structure (capture on target host)

SCREENSHOT PLACEHOLDER — NOT CAPTURED

Insert a genuine Ubuntu terminal capture for this figure. See README_Execution_Guide.md for the exact command.

Figure 4.3: 4.3 Installation process (capture on target host)

SCREENSHOT PLACEHOLDER — NOT CAPTURED

Insert a genuine Ubuntu terminal capture for this figure. See README_Execution_Guide.md for the exact command.

Figure 4.4: 4.4 Disk monitoring output (capture on target host)

SCREENSHOT PLACEHOLDER — NOT CAPTURED

Insert a genuine Ubuntu terminal capture for this figure. See README_Execution_Guide.md for the exact command.

Figure 4.5: 4.5 Inode monitoring output (capture on target host)

SCREENSHOT PLACEHOLDER — NOT CAPTURED

Insert a genuine Ubuntu terminal capture for this figure. See README_Execution_Guide.md for the exact command.

Figure 4.6: 4.6 Storage history and growth sample (capture on target host)

SCREENSHOT PLACEHOLDER — NOT CAPTURED

Insert a genuine Ubuntu terminal capture for this figure. See README_Execution_Guide.md for the exact command.

Figure 4.7: 4.7 Warning and critical alert output (capture on target host)

SCREENSHOT PLACEHOLDER — NOT CAPTURED

Insert a genuine Ubuntu terminal capture for this figure. See README_Execution_Guide.md for the exact command.

Figure 4.8: 4.8 Generated text report (capture on target host)

SCREENSHOT PLACEHOLDER — NOT CAPTURED

Insert a genuine Ubuntu terminal capture for this figure. See README_Execution_Guide.md for the exact command.

Figure 4.9: 4.9 Cron configuration and execution log (capture on target host)

SCREENSHOT PLACEHOLDER — NOT CAPTURED

Insert a genuine Ubuntu terminal capture for this figure. See README_Execution_Guide.md for the exact command.

Figure 4.10: 4.10 Timestamped backup creation (capture on target host)

SCREENSHOT PLACEHOLDER — NOT CAPTURED

Insert a genuine Ubuntu terminal capture for this figure. See README_Execution_Guide.md for the exact command.

Figure 4.11: 4.11 Compressed archive integrity check (capture on target host)

SCREENSHOT PLACEHOLDER — NOT CAPTURED

Insert a genuine Ubuntu terminal capture for this figure. See README_Execution_Guide.md for the exact command.

Figure 4.12: 4.12 Selected-file recovery (capture on target host)